

Formation FPGA

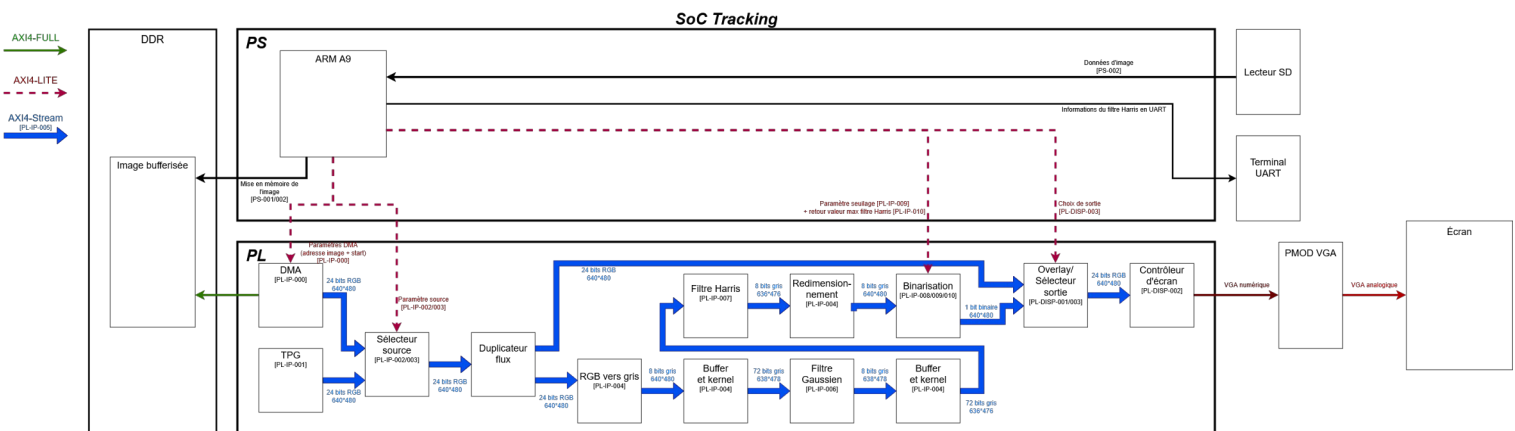
Projet SoC Tracking

Rapport de validation

Introduction

Ce rapport décrit notre implémentation et notre validation des différentes spécifications pour le projet SoC Tracking. Il suit le plan de validation qui fait partie des autres rendus de ce projet.

En récapitulant tous blocs nécessaires pour atteindre toutes les spécifications du projet selon les descriptions que nous avons données dans le plan de validation, l'architecture globale de notre produit doit être la suivante (l'image est aussi disponible séparément pour être plus lisible) :



Entrées/sorties

Cette première partie se concentre sur la validation des fonctionnalités de génération de signal vidéo AXI4-Stream, ainsi que leur affichage sur un écran.

[PL-IP-005] Standards de transferts [1]

Comme évoqué dans le plan de validation, cette première spécification n'a pas vraiment à être validée en soit, puisqu'elle guide simplement la conception de notre architecture.

En pratique, nous avons suivi cette spécification en utilisant le format AXI4-Stream pour l'ensemble de nos flux vidéo à l'intérieur du FPGA, de leur génération jusqu'à leur conversion de sortie au format VGA.

Cette spécification [PL-IP-005] est donc **validée**

[PL-IP-001] Source Vidéo TPG [1]

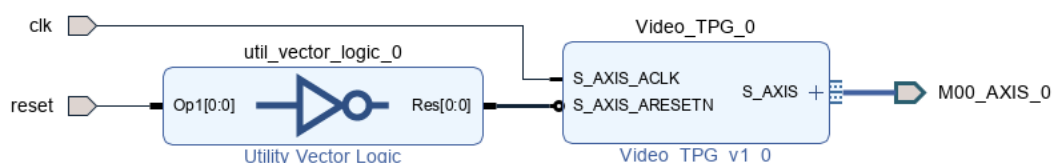
Deux pistes étaient envisagées pour la réalisation de cette fonctionnalité :

- l'IP constructeur Xilinx « Video Test Pattern Generator », listée dans le catalogue d'IP de Vivado,
- le code VHDL fournis par Digilent dans les ressources « Cora-Z7-10-Pmod-VGA-2018.2-1 » fournies afin de faire fonctionner l'adaptateur Pmod VGA de Digilent pour carte Cora Z7, qui contient une partie générant les signaux RGB selon un schéma de test prédéfini qui correspond à notre spécification.

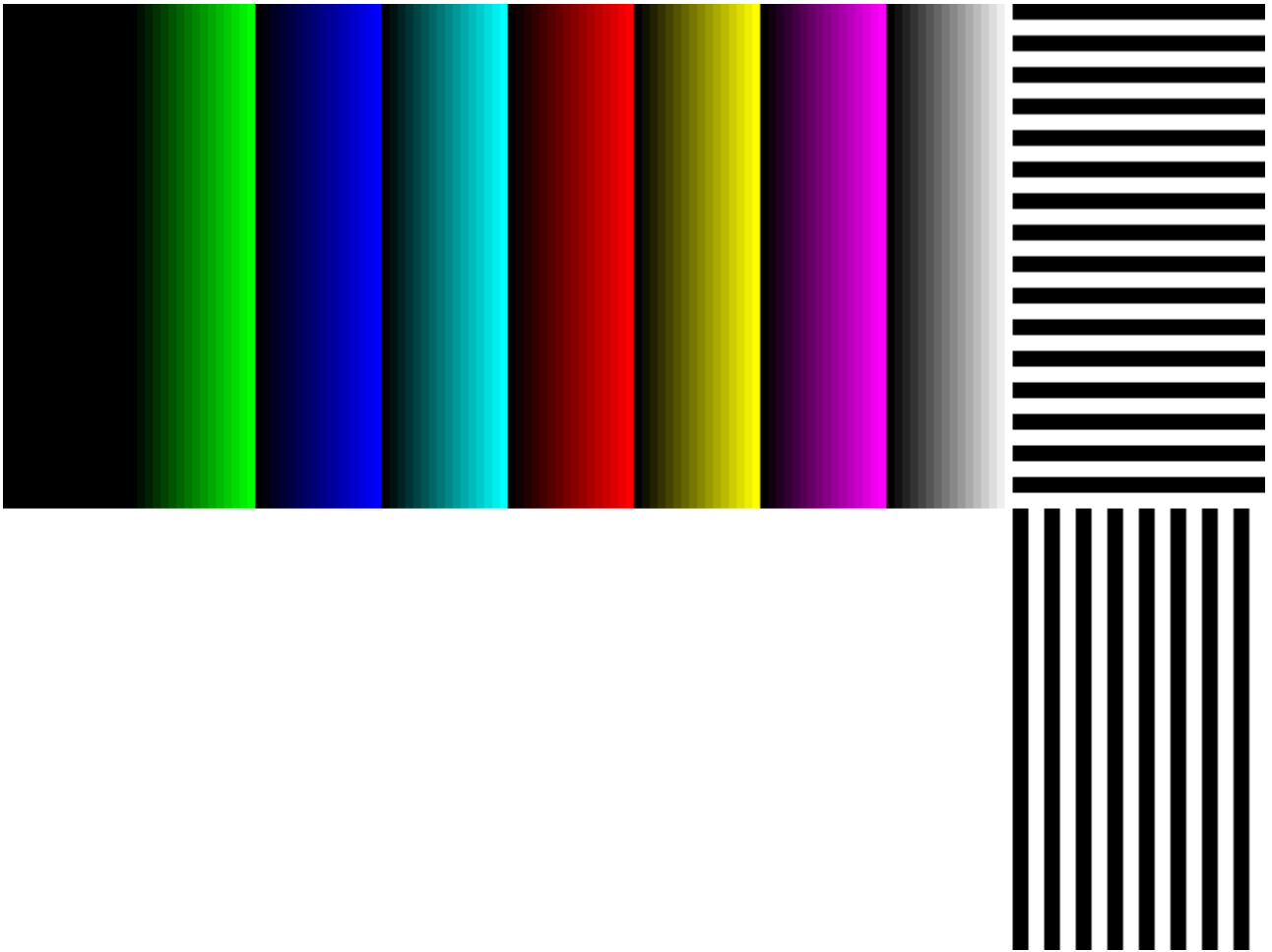
En pratique, l'IP Xilinx « Video Test Pattern Generator » souffre d'un problème dans notre version de Vivado 2020.2, et était impossible à utiliser. Nous avons donc implémenter notre IP **Video_TPG** en partant de « Cora-Z7-10-Pmod-VGA-2018.2-1 », qui propose déjà un code VHDL générant un schéma vidéo de test. Nous l'avons adapté pour lui ajouter une sortie Master AXI4-Stream, conditionnant le calcul et la mise à disposition du pixel suivant à un 'handshake' (TVALID='1' et TREADY='1') sur la donnée de pixel actuelle.

Inspection : notre code VHDL est une modification d'un code préexistant qui fonctionne. Les principales modifications concernent des gestions de timing simple (handshake AXI4-Stream).

Simulation : On crée un testbench *tb_tpg.vhd*.



La sortie de notre TPG est enregistrée sur un moniteur RGB (*Axi4s_monitor_RGB.vhd*) qui enregistre chacun des canaux de couleur et permet de reconstituer l'image de sortie :



Cette image est identique à celle qui s'affichait à l'écran avec « Cora-Z7-10-Pmod-VGA-2018.2-1 ». Notre **simulation peut donc être validée**.

Démonstration: une fois la fonctionnalité d'affichage VGA également implémentée et simulée (voire fonctionnalité suivante), les deux blocs pourront être démontrés conjointement, avec affichage sur écran de notre TPG.

[PL-DISP-002] Affichage écran VGA [1]

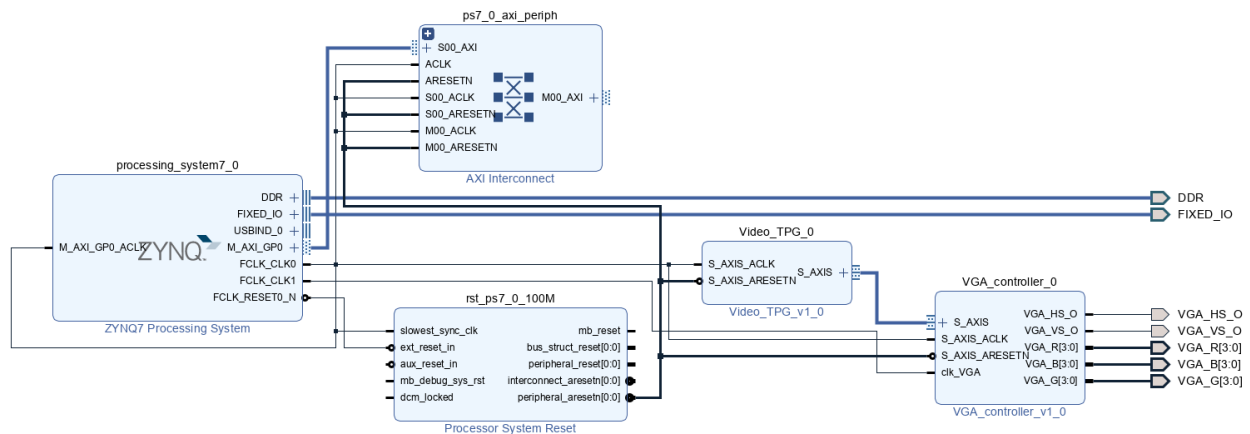
Nous sommes encore une fois repartis de « Cora-Z7-10-Pmod-VGA-2018.2-1 », en reprenant cette fois la partie du code qui effectue le cadencage des canaux RGB et la génération des signaux de synchronisation horizontaux et verticaux pour de l'affichage VGA numérique. Nous avons interfacé ce code avec une lecture de signaux RGB depuis un AXI4-Stream Slave, pour constituer l'IP **VGA_controller**.

Deux horloges sont utilisées dans cette IP : l'une gère la composante AXI4-Stream, à 100 MHz, l'autre la partie VGA à 25 MHz. Pour cadencer l'arrivée des données, c'est l'horloge VGA qui lève

le TREADY quand il a besoin d'une nouvelle valeur, et l'horloge AXI qui lance un reset_asynchrone après un 'handshake' pour le faire redescendre.

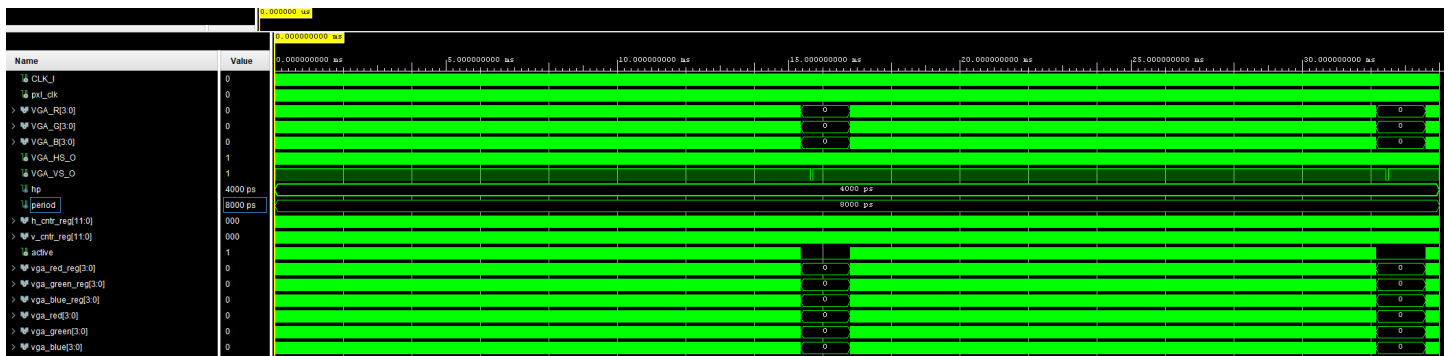
Inspection : notre code VHDL est une modification d'un code préexistant qui fonctionne. Les principales modifications concernent l'implémentation d'un AXI4-Stream.

Simulation : on crée un testbench tb_tpg_vga.vhd qui regroupe notre IP TPG et notre IP VGA.

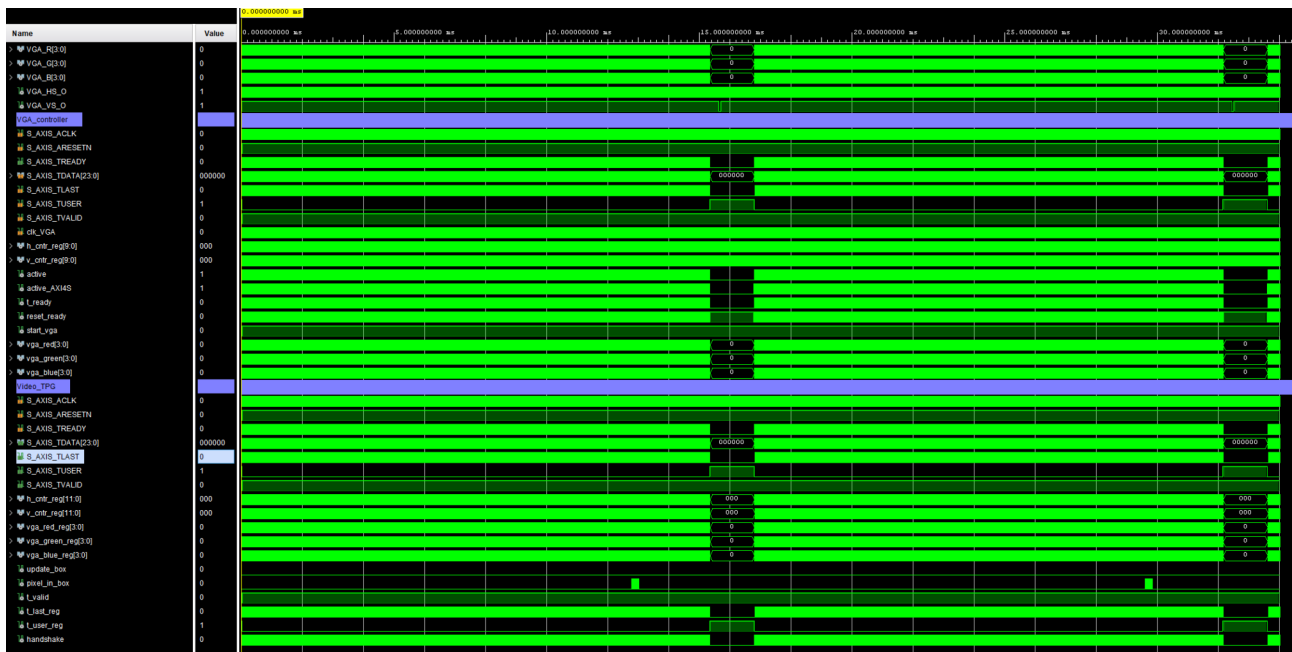


Cette simulation est comparé à celui obtenu pour le projet « Cora-Z7-10-Pmod-VGA-2018.2-1 ».

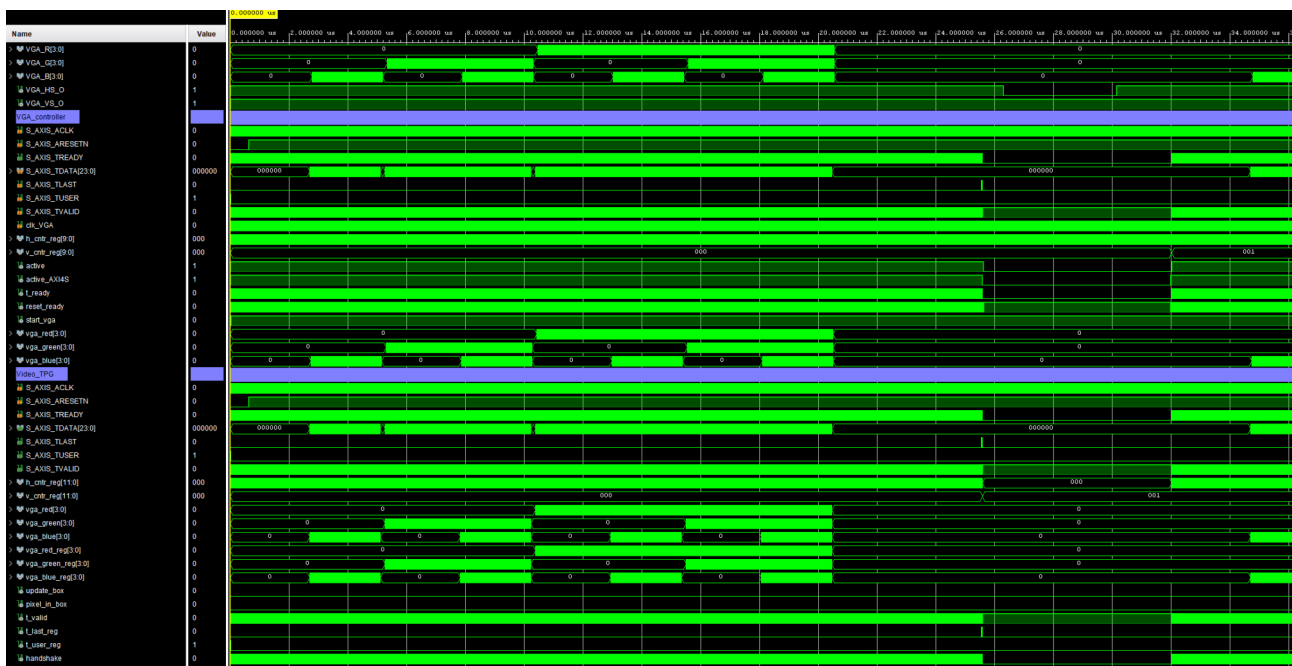
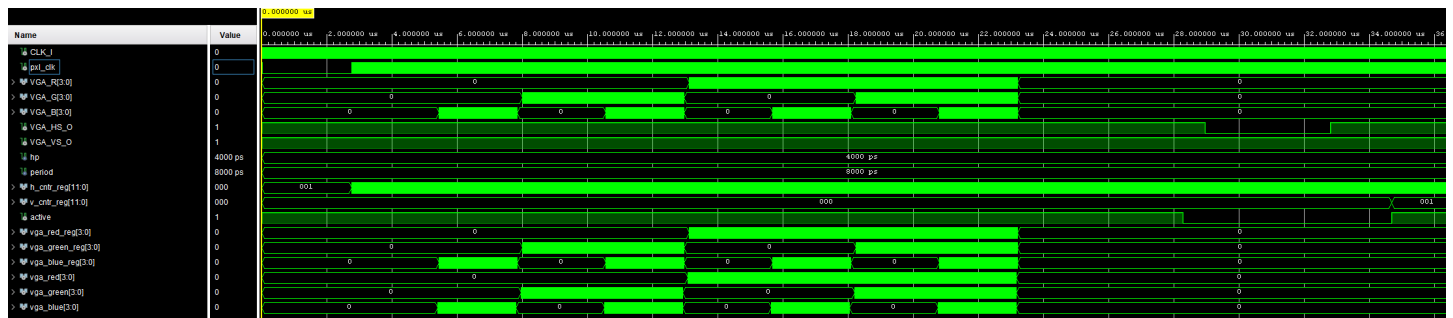
Ainsi, au global, on avait pour PMOD_VGA :



Et pour notre simulation TPG_VGA :



Si on zoom sur la période correspondant à la première ligne de l'image, on observe respectivement :



Les progressions semblent identiques sur cet extrait. En notant les timings de changement de lignes, d'image, et les changements de polarités des signaux de synchronisation horizontale et verticale, on observe exactement les mêmes délais entre notre simulation et celle du module fonctionnel PMOD_VGA. Notre **simulation peut donc être validée**.

Démonstration : On programme notre carte avec l'architecture décrite pendant la démonstration. 5 secondes de vidéo de la démonstration sont visibles dans *TPG_VGA.mp4*. On observe bien la même vidéo que celle obtenue avec « Cora-Z7-10-Pmod-VGA-2018.2-1 ». Notre **démonstration peut donc être validée**.

Au global ces étapes de validations nous permettent d'affirmer que *[PL-IP-001]* et *[PL-DISP-002]* sont validés

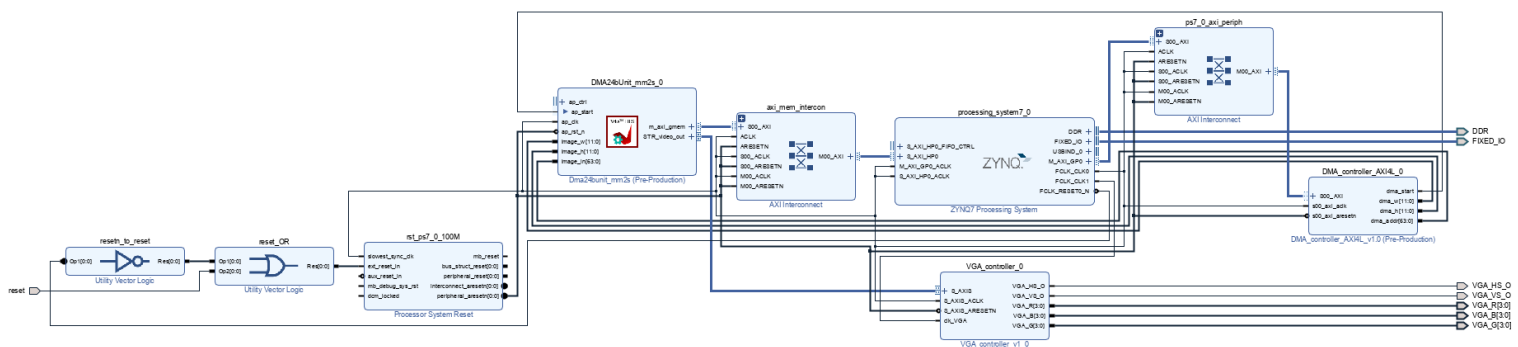
[PL-IP-000] Source Vidéo mémoire [1] et [PS-001] Mise en mémoire manuelle pattern [1]

En plus de l'IP « DMA_RGB24b » déjà fournie, nous avons créé un bloc de contrôle de cette DMA, l'IP **DMA_controller_AXI4L_1.0** qui fournit à la DMA :

- par paramètres génériques fournis à l'instanciation, les dimensions de l'image ;
- par registre piloté par la partie PS via une connexion AXI4-Lite, l'adresse de l'image dans la DDR et le déclenchement de la lecture.

Inspection : la plupart du code, tout celui de « DMA_RGB24b », est déjà validé. Nos ajouts concernent une simple instance d'AXI4-Lite, et le transfert de 4 signaux, dont 2 simplement lus depuis un paramètre générique, et 2 lus depuis des registres.

Démonstration : on a construit une architecture comportant la partie PS, le lecteur DMA et son contrôleur, ainsi que le contrôleur d'affichage VGA.



On teste cette architecture via Vitis, en utilisant la console XSCT pour charger la mémoire DDR et configurer les registres du contrôleur VGA.

```
xsct% mwr -bin -file sample_in_rgb.bin 0x10000000 307200
xsct% mwr 0x44A00000 0x10000000
xsct% mwr 0x44A00008 1
xsct%
```

Notre image apparaît bien sur notre écran, comme on peut le voir sur la vidéo *DMA_VGA.mp4* : notre **démonstration peut être validée**.

Au global ces étapes de validations nous permettent d'affirmer que *[PL-IP-000]* et *[PS-001]* sont **validés**

Détection de contours

***[PL-IP-004]* Architecture filtre à convolution [1]**

Pour passer nos AXI4-Stream initiaux de RGB à une nuance de gris, nous avons conçu l'IP **AXI4_Stream_RGB_to_grey_1.0**.

Inspection : notre code consiste en un simple changement à la volée de la valeur des données d'un AXI4-Stream, sans se soucier des autres signaux ou devoir changer les transmission. Une attention particulière aura été portée pour s'assurer qu'il n'y a pas d'effet d'arrondi ou de seuil lors du calcul de la moyenne des 3 canaux RGB.

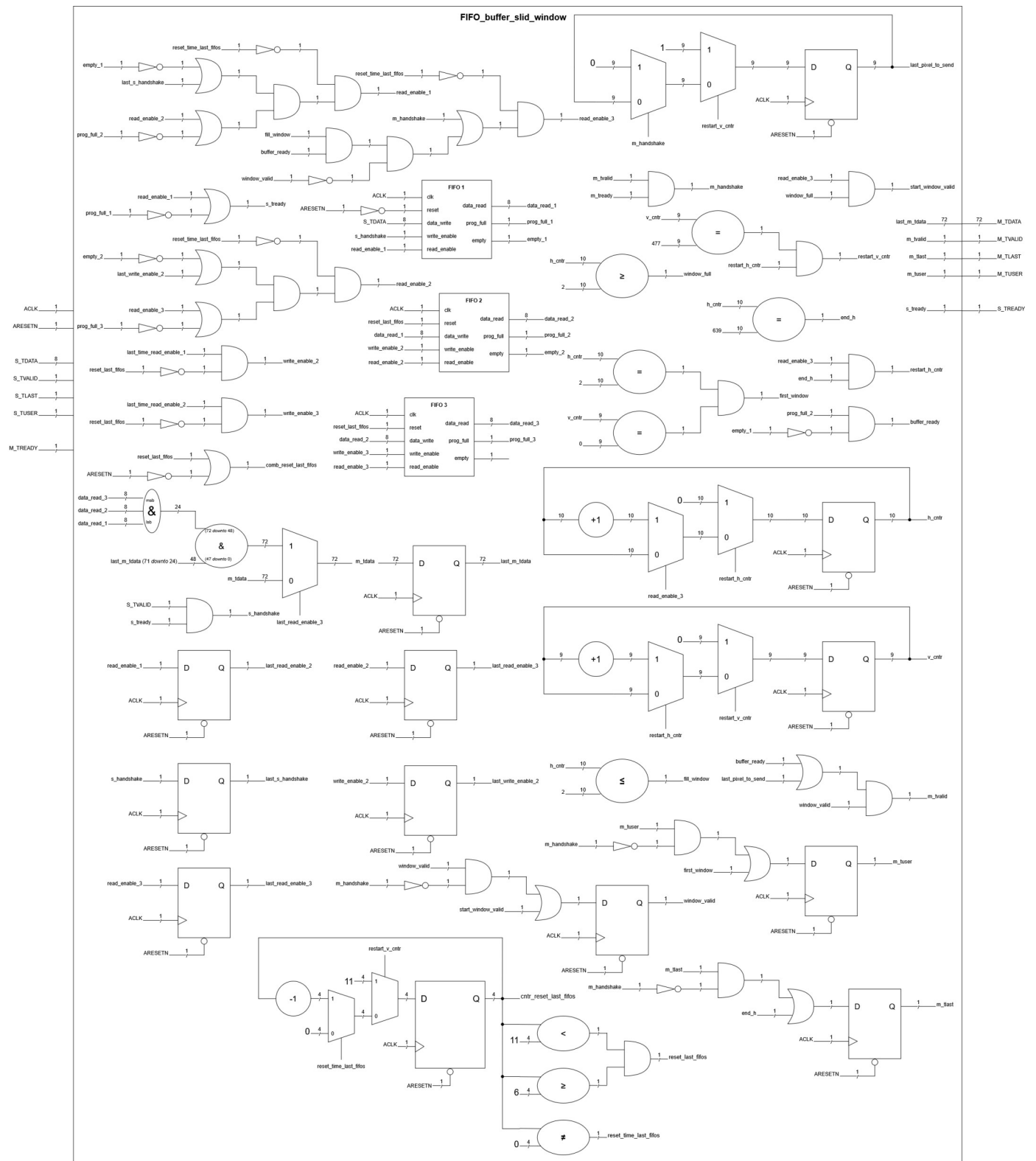
Simulation : on construit un testbench *tb_RGB_to_grey.vhd*, où un driver (*Axi4s_driver_RGB.vhd*) lit les différents canaux d'une image RGB pour alimenter notre bloc, dont la sortie est ensuite fournie à un moniteur (*Axi4s_monitor.vhd*) qui enregistre l'image produite.

Cette image (à gauche) est identique à celle que produit ImageJ quand on lui demande de convertir en niveaux de gris l'image originale (à droite). Notre **simulation peut donc être validée**.



Ensuite, nous avons réalisé l'IP **AXI4S_fifo_buffer_slid_window_1.0** qui peut stocker un buffer de trois lignes à la fois de l'image en cours, pour en extraire une fenêtre glissante de trois pixels par trois, le kernel permettant ultérieurement d'appliquer un filtre.

Inspection : notre IP est expliquée par le schéma RTL suivant (l'image est disponible séparément pour offrir une meilleure lisibilité) :



L'architecture comporte 3 fifos qui servent de buffer : chacune peut stocker 640 pixels, et une fifo se vide dans la suivante, ainsi un même pixel parcourra en général les 3 fifos successivement. Les 3 fifos sont synchronisées : en fonctionnement normal, des pixels de la même colonne sont lus simultanément sur les 3 fifos.

La sortie, c'est à dire le kernel de 3*3, est constitué en ajoutant la colonne obtenu après lecture simultanée des 3 fifos, côté msb, et en décalant les autres colonnes déjà présentes dans le précédent kernel. L'IP connaît la position du kernel dans l'image, et en déduit les signaux TLAST et TUSER.

L'IP gère aussi le reset des deux dernières fifo en fin d'image, la première fifo contenant à ce moment les informations de la première ligne de la nouvelle image.

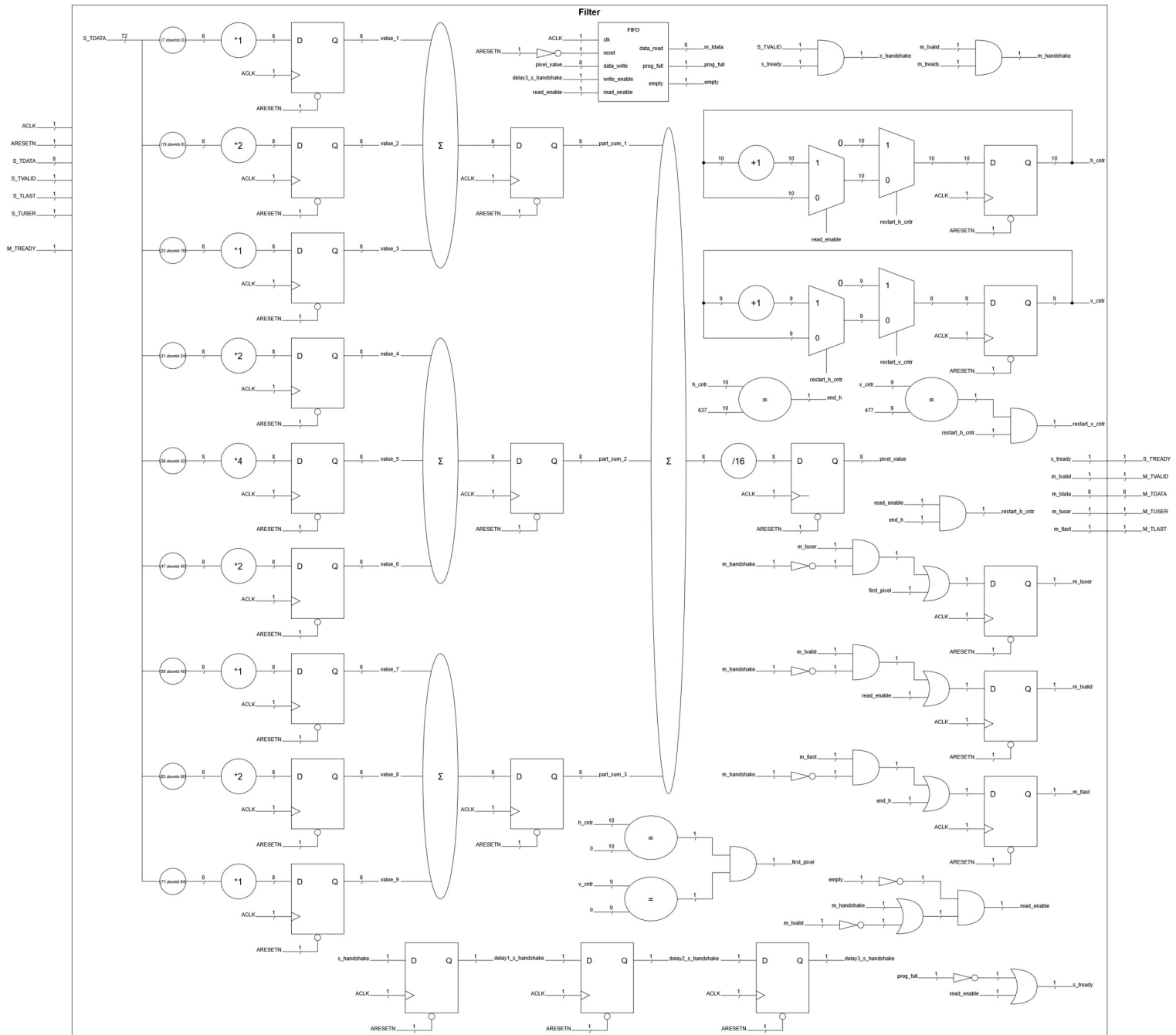
Simulation : on construit un testbench *tb_fifo_buf_slid_window.vhd*, où un driver (*Axi4s_driver.vhd*) lit plusieurs fois une image en niveaux de gris pour alimenter notre bloc, dont la sortie est ensuite fournie à un moniteur qui enregistre le flux produit (en convertissant en entier relatif la valeur des 72 bits du flux de sortie, et le stockant comme un unique pixel). La suite de valeurs enregistrée en sortie n'a pas de sens en tant qu'image, mais le fait qu'elle soit toutes identiques permet de confirmer la constance de la solution (il n'y a pas de décalage, le bloc produit bien pour chaque image un nombre de fenêtres correspondant à deux lignes et deux colonnes de moins que l'image d'origine).

La validation de ce bloc continue avec la spécification suivante.

[PL-IP-006] Filtre gaussien [1] et [PL-IP-007] Filtre Harris [1]

Pour réaliser les deux filtres, gaussien et Harris, nous avons conçu une unique IP **AXI4S_Filter_1.0**, dont la matrice de coefficients est entièrement paramétrable pour chaque instantiation du filtre.

Inspection : notre IP est expliquée par le schéma RTL suivant (l'image est disponible séparément pour offrir une meilleure lisibilité) :



Notre IP applique le filtre en trois temps (pour des questions de timing) : elle calcule d'abord la multiplication par son coefficient pour chaque pixel du kernel, puis au coup d'horloge suivant calcule trois sommes partielles de trois valeurs, pour au coup suivant faire la somme finale et la division et stocker cette valeur filtrée dans une fifo.

La fifo distribue ces valeurs en donnée de sortie au fur et à mesure des transactions confirmées.

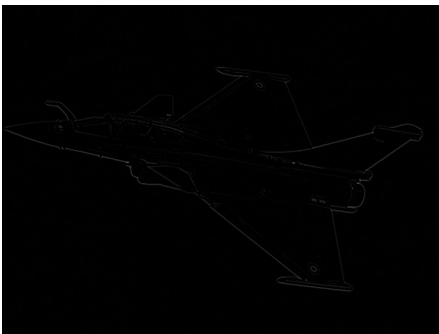
Simulation : on construit deux testbench *tb_filter_Gaussian.vhd* et *tb_filter_Harris.vhd*, où un driver (*Axi4s_driver.vhd*) lit plusieurs fois une image en niveaux de gris pour alimenter notre bloc buffer, dont la sortie de kernels est reçue par le filtre qui a son tour transmet son flux de sortie à un moniteur (*Axi4s_monitor.vhd*) qui enregistre l'image produite. Cette image doit être identique au

résultat que donne ImageJ pour une convolution de l'image d'origine par la même matrice de coefficient. On compare aussi les images de sorties successives qui doivent être identiques.

Voici respectivement la première image simulée, la seconde, et le modèle issu d'ImageJ pour le filtre Gaussien, elles sont toutes identiques :



De même, voici respectivement la première image simulée, la seconde, et le modèle issu d'ImageJ pour le filtre Gaussien, elles sont toutes identiques :



Notre **simulation peut donc être validée.**

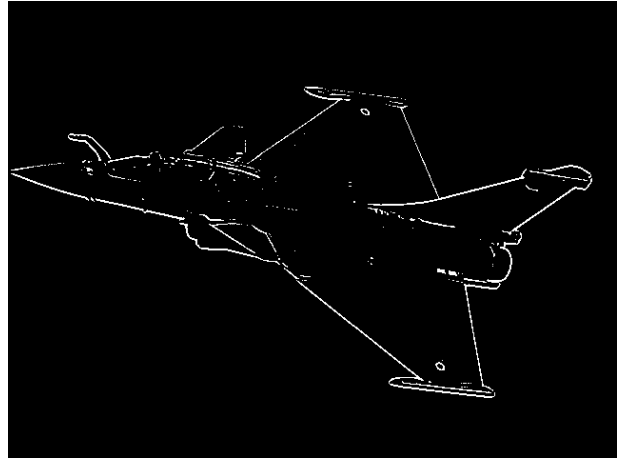
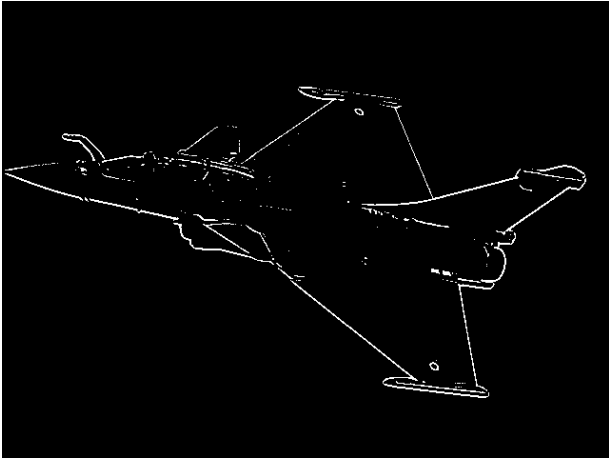
[PL-IP-008] Filtre Harris binarisation [1]

Nous avons conçu une IP **AXI4S_Binarisation_simple_1.0** qui effectue une binarisation du flux d'images AXI4-Stream, en fonction d'un seuil paramétrable à l'instanciation du bloc.

Inspection : le code effectue la transformation à la volée de la donnée TDATA d'un AXI4-Stream, sans changer les autres canaux.

Simulation : on construit un testbench, où un driver (*Axi4s_driver.vhd*) lit une image en niveaux de gris pour alimenter notre bloc de binarisation, dont la sortie est transmise à un moniteur (*Axi4s_monitor.vhd*) qui enregistre l'image produite. L'image produite doit être identique au résultat que donne ImageJ pour une binarisation de l'image d'origine par le même seuil.

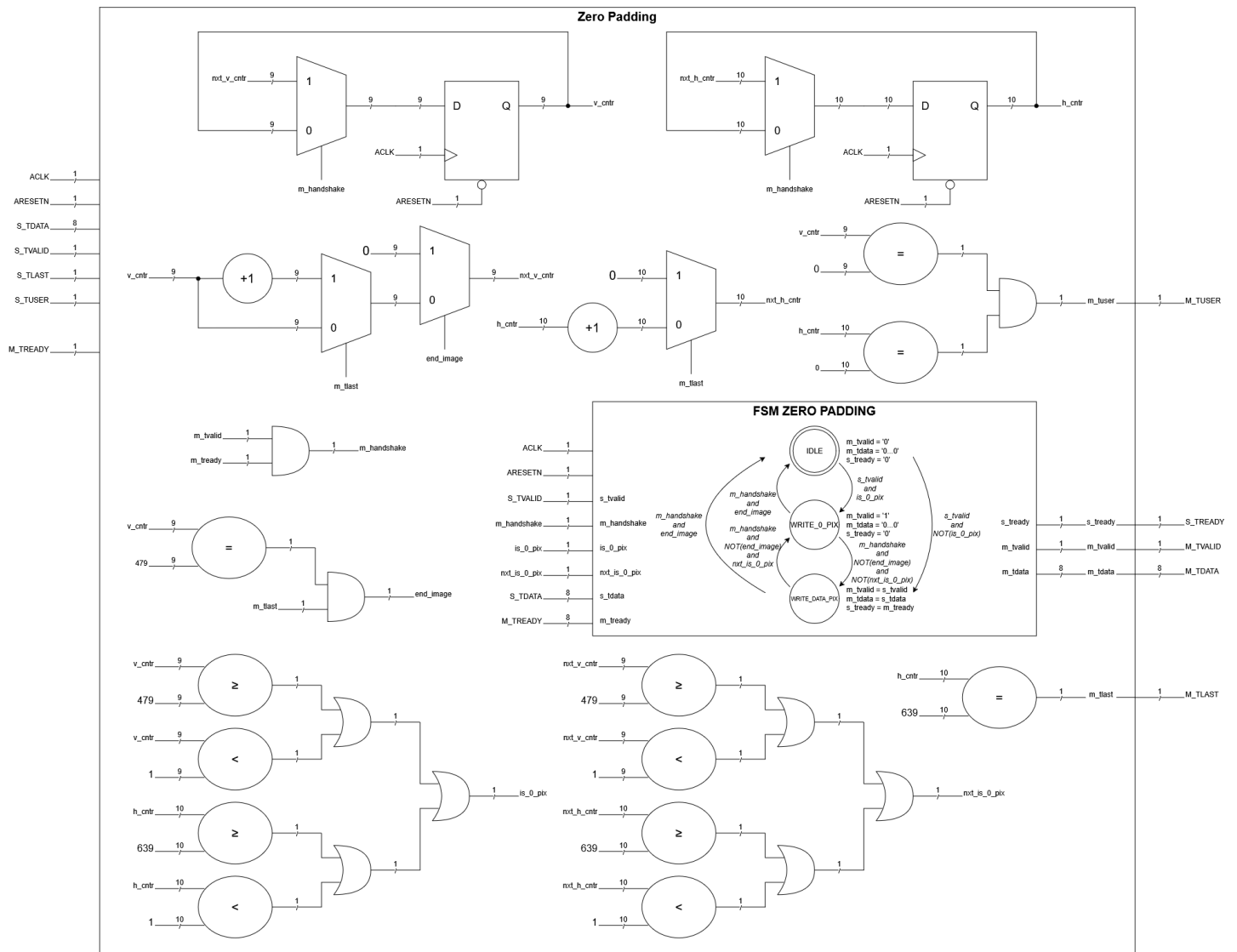
Voici l'image issue de la simulation et celle d'ImageJ, elles sont identiques :



Notre **simulation** peut donc être validée.

En complément nous élaborons l'IP **AXI4S_zero_padding_1.0** dont la fonction est de remplir les colonnes et lignes extérieures d'une image de zéros, pour augmenter les dimensions de l'image. Le nombre de lignes et de colonnes à ajouter de chaque côté sont paramétrable à l'instanciation.

Inspection : notre IP est expliquée par le schéma RTL suivant (l'image est disponible séparément pour offrir une meilleur lisibilité) :



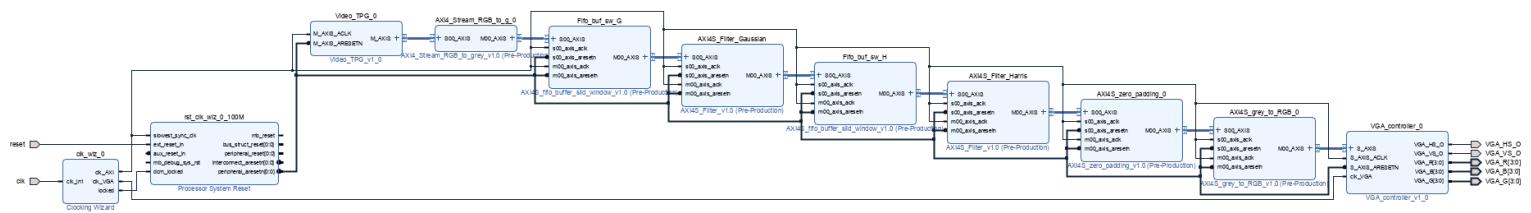
Notre IP est dirigée par une FSM, qui, une fois qu'elle sait que ses données d'entrées sont valides, va déterminer en fonction de sa position dans l'image si elle est dans la zone extérieure de zero-padding, et renvoyer une valeur de '0', ou pas et transmettre la donnée en entrée.

Simulation : on construit un testbench *tb_zero_padding.vhd*, où un driver (*Axi4s_driver.vhd*) lit une image en niveaux de gris pour alimenter notre bloc zéro-padding, dont la sortie est transmise à un moniteur (*Axi4s_monitor.vhd*) qui enregistre l'image produite. L'image produite doit contenir l'image de départ, entourée de pixels noirs.

En pratique, l'image obtenue en sortie de simulation est bien construite, et notre **simulation peut donc être validée**.

En ajoutant un bloc qui transforme un flux AXI4-Stream en niveau de gris en flux AXI4-Stream RGB (chaque canal prend la valeur de niveau de gris), on a tous les composants pour appliquer la détection de contour et revenir à un flux affichable en VGA.

On peut donc mettre en place une première **validation** groupée de nos composants de détection de contour par démonstration, en faisant fonctionner une architecture :



(Le bloc de binarisation n'est pas inclus, des valeurs uniquement de '0' et '1' serait indistinguable en VGA.)

Le résultat de cette démonstration est présenté dans la vidéo *TPG_banc_simple_VGA.mp4*.

Cette démonstration n'est pas une réussite : notre vidéo se décale continuellement vers la gauche, et très lentement vers le haut, suggérant qu'un ou plusieurs pixels manquent dans chaque image.

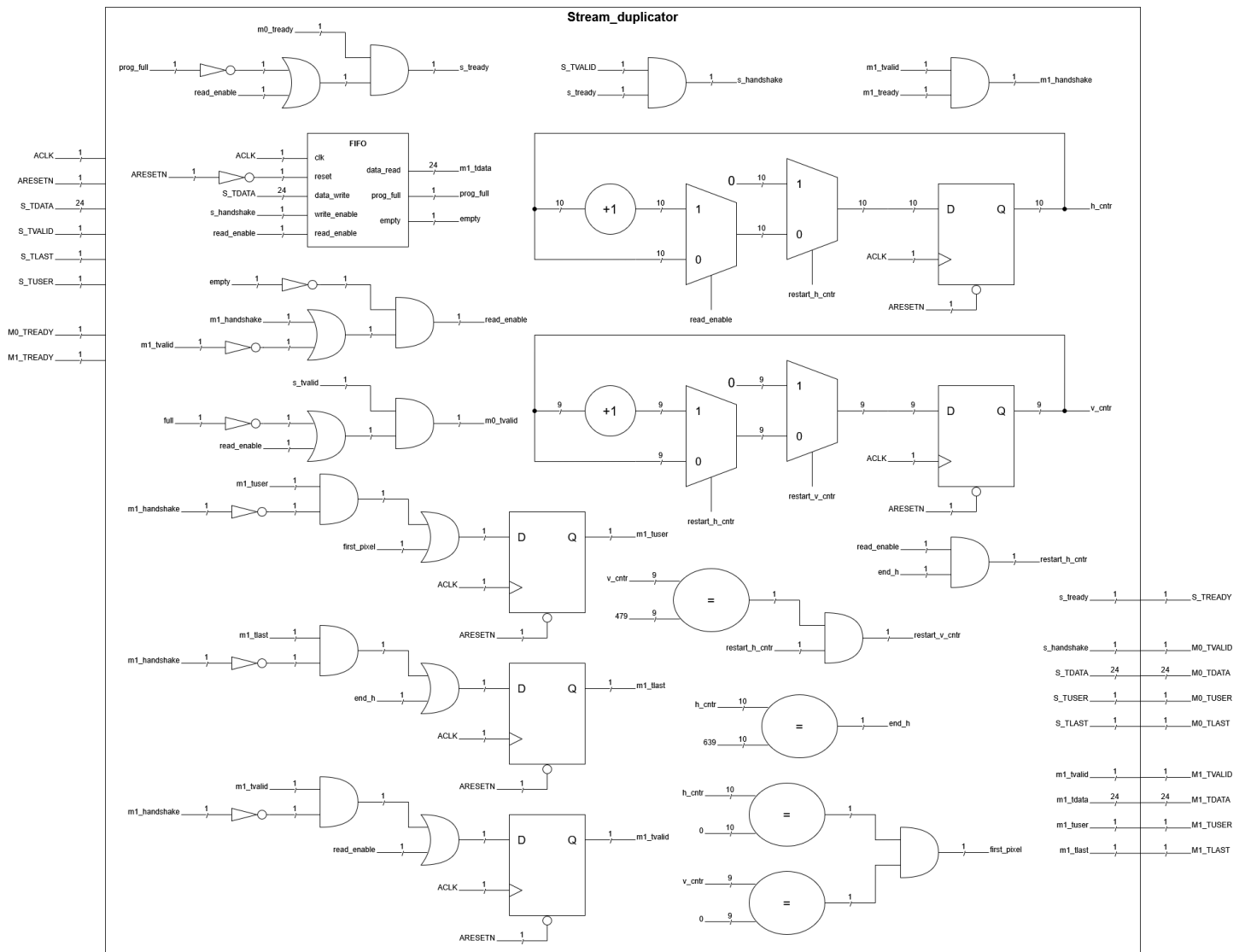
La différence avec ce que l'on a observé sans problème en simulation vient probablement de la contrainte fixe de réception d'un pixel tous les 4 coups d'horloge, ce qui inclut des variations de timings qui n'étaient pas présentes dans un contexte d'acquisition continue.

En l'état, **cette architecture n'est pas validée**, mais nous préférons continuer à implémenter le reste avant de revenir investiguer et régler ce décalage.

[PL-DISP-001] Overlay [1]

Nous réalisons l'IP **AXI4S_Stream_Duplicator_1.0** capable de dupliquer un flux AXI4-Stream : le flux d'entrée est stocké dans le bloc, une sortie le restitue immédiatement (et reste synchronisée sur l'entrée) alors que l'autre peut le restituer avec un décalage.

Inspection : notre IP est expliquée par le schéma RTL suivant (l'image est disponible séparément pour offrir une meilleur lisibilité) :



Le stockage du flux vidéo est effectuée dans une fifo. Celle-ci est dimensionnée avec une profondeur de 4096, afin de garantir un décalage possible le temps du reste des traitements de détection de contour, principalement le stockage de 4 lignes entières (~640*4 pixels) dans les fifos des buffer/kernel.

Simulation : on construit un testbench *tb_duplication.vhd*, où un driver lit une image en niveaux de gris pour alimenter notre bloc de duplication, dont les sorties sont reliées à deux moniteurs qui enregistrent les images produites. Pour vérifier le décalage du deuxième flux de sortie, le moniteur correspondant aura une période de reset initial bien plus longue que le reste des composants de l'architecture. Les deux images produites, ainsi que l'image de départ, doivent être identiques.

C'est bien ce qu'on observe en pratique, notre **simulation peut donc être validée**.

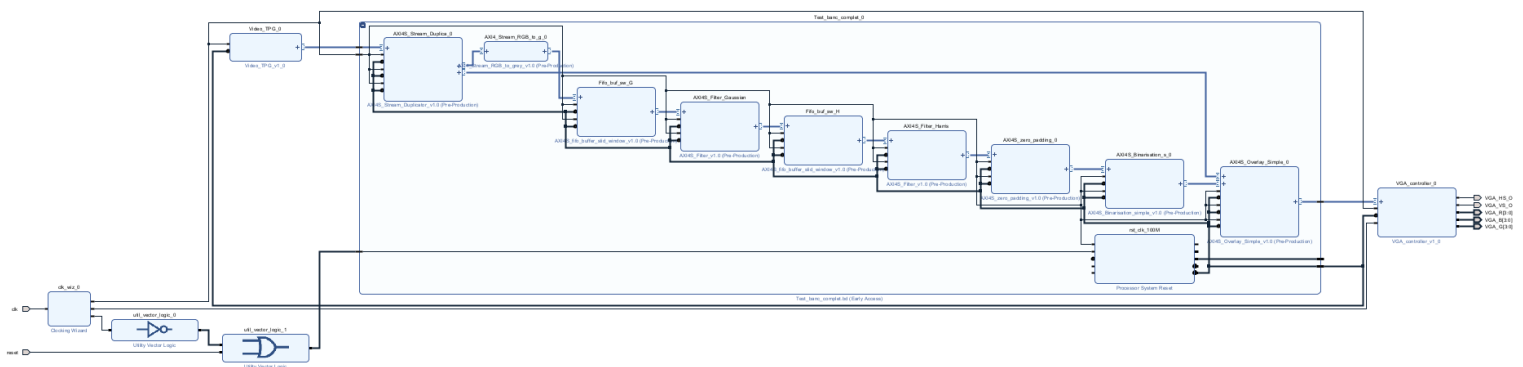
Dernière IP pour la détection de contour : **AXI4S_Overlay_simple_1.0** qui superpose la détection de contour à l'image d'origine. Un trio de paramètres RGB, choisis à l'instanciation du bloc, détermine la couleur sous laquelle apparaissent les pixels à '1' dans le flux binarisé, les autres pixels apparaissent avec leur valeur dans l'image d'origine.

Inspection : le code fait de la modification à la volée du TDATA d'un flux AXI4-Stream, ce qui ne demande pas de gestion des timings.

Simulation: on construit un testbench *tb_overlay.vhd*, où un driver lit une image RGB, et un autre lit une image binarisée, et les deux alimentent l'overlay, dont la sortie est reliée à un moniteur qui enregistre l'image produite.

C'est bien ce qu'on observe en pratique, notre **simulation peut donc être validée**.

On peut donc mettre en place une deuxième **validation** groupée de nos composants de détection de contour par démonstration, en faisant fonctionner une architecture :



Le résultat de cette démonstration est présenté dans la vidéo *TPG_banc_VGA.mp4*.

Le résultat est étrange : le fond de l'image est fixe, mais l'overlay commence à se décaler vers la gauche après le reset (comme dans la démonstration précédente). Cependant, après quelques secondes, l'overlay se stabilise, et garde un décalage horizontal constant avec l'image dont il décrit les contours.

La démonstration n'est pas une réussite et l'architecture n'est pas vraiment validée, mais elle offre un contexte stable qui suffira pour valider les fonctionnalités suivantes.

Paramétrisation

[PL-IP-002] Sélection de source vidéo init [1]

On conçoit l'IP **Input_selector_AXI4L_1.0** qui dispose de deux entrées et une sortie AXI4-Stream, ainsi qu'un Slave AXI4-Lite piloté par le PS. Le bloc a deux bits de registre : le premier indique la source (TPG ou DMA) qu'on retrouve en sortie, le second indique que l'initialisation est faite,

verrouillant la source et déclenchant le flux. Après verrouillage, modifier le premier bit de registre n'a aucun effet.

Inspection : le code VHDL est un simple transfert d'AXI4S-Stream sans gestion des timings, avec une sélection de flux en fonction du registre AXI4-Lite.

Test : on reprend l'architecture complète en ajoutant la partie PS et la nouvelle IP sélecteur de source vidéo. On teste cette architecture via Vitis, en utilisant la console XSCT pour d'abord définir la valeur du registre choisissant la source, puis celle qui confirme la source et démarre le flux.

Sur la vidéo *Input_init.mp4*, on voit bien l'écran afficher d'abord le TPG, puis après un reset le DMA.

[PL-IP-002] est validée.

[PL-IP-003] Sélection de source vidéo à chaud [2]

On modifie **Input_selector_AXI4L_1.0** pour qu'un changement du premier bit de registre puisse toujours effectuer un changement de source, mais que celui-ci ne se produise qu'au moment du premier pixel d'une image (signal AXI4-Stream TUSER). Le flux non-utilisé restant figé, il attend toujours à un premier pixel : soit parce que le flux n'a pas encore été utilisé depuis l'initialisation, soit parce qu'il s'est arrêté sur ce pixel au dernier changement de source. Le second bit du registre n'a plus d'effet de verrou, simplement de démarrage global du stream.

Test : on reprend l'architecture précédente avec l'IP modifiée. On teste cette architecture via Vitis, en utilisant la console XSCT pour d'abord démarrer le flux, puis changer successivement de source. Sur la vidéo *Input_live.mp4*, on voit qu'à chaque commande de changement de source, l'écran affiche bien la source correspondante.

[PL-IP-003] est validée.

[PL-IP-009] Filtre Harris seuillage paramétrable [1]

On modifie l'IP de binarisation qui devient **AXI4S_Binarisation_AXI4L_1.0** pour lui ajouter un Slave AXI4-Lite piloté par le PS. Le bloc possède un registre, où le PS indique la valeur du seuil de binarisation. Le calcul dans le bloc utilise la valeur du registre, au lieu d'un paramètre défini à l'instanciation du bloc.

Test : on reprend l'architecture précédente avec l'IP modifiée. On teste cette architecture via Vitis, en utilisant la console XSCT pour enregistrer dans le registre différentes valeurs de seuil.

Dans la vidéo *Seuil_live.mp4*, à chaque valeur de seuil, l'écran affiche bien l'overlay mis à jour : plus le seuil est bas, plus il y a de pixels mis en avant sur l'overlay.

[PL-IP-009] est validée.

[PL-DISP-003] Mode d'affichage [2]

On modifie l'IP d'overlay qui devient **AXI4S_Overlay_AXI4L_1.0** en lui ajoutant un Slave AXI4-Lite piloté par le PS. Le bloc possède un registre d'un bit, où le PS indique le mode d'affichage. En fonction du mode d'affichage, le bloc laisse sortir l'image superposée de son overlay, ou bien seulement l'image. Pour éviter de créer un décalage entre les deux sources du bloc overlay, on continue de synchroniser l'entrée des deux flux dans le bloc, même si l'overlay n'est pas utilisé en sortie.

En plus de cela, on rajoute une valeur de registre qui décide en live de la couleur de l'overlay

Test : on reprend l'architecture précédente avec l'IP modifiée. On teste cette architecture via Vitis, en utilisant la console XSCT pour alterner dans le registre entre les deux modes d'affichage, et essayer différentes couleurs d'overlay.

Sur la vidéo *Overlay_live.mp4*, on remarque bien qu'à chaque changement de valeur, l'overlay s'ajoute ou disparaît de l'image sur l'écran, et l'overlay passe ensuite du cyan au jaune au magenta.

[PL-DISP-003] est validée.

[PS-002] Mise en mémoire automatique pattern [2]

Je n'ai pas eu le temps de configurer la lecture automatique de l'image depuis la carte SD.

En revanche, j'ai mis en place un fichier .c qui contient l'ensemble des valeurs de *sample_in.bin*.

Ce fichier est intégré à notre firmware, et automatiquement utilisé pour programmer l'image dans la DDR.

Test : On teste le nouveau firmware avec l'architecture complète via Vitis.

L'image sur l'écran doit dès la mise en route correspondre à l'image programmée dans le firmware.

Notre **solution alternative est validée**, mais la spécification originale **[PS-002]** ne l'est pas.

Spécifications de retours

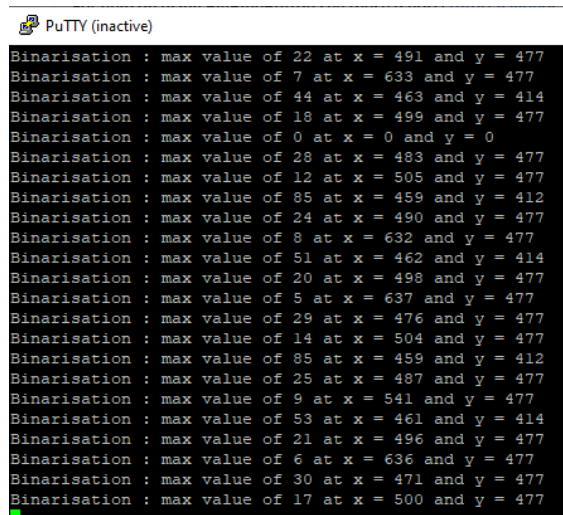
[PL-IP-010] Filtre Harris retour de données [2] et **[PS-003]** Retour de données UART [2]

On modifie à nouveau le fonctionnement de l'IP **AXI4S_Binarisation_AXI4L_1.0**, pour qu'elle calcule sur chaque image la valeur maximale de son entrée (le résultat du filtre Harris) et la position de ce maximum. On modifie le fonctionnement du registre AXI4-Lite Slave pour, en plus de recevoir depuis le PS la valeur du seuil de binarisation, permettre au PS de lire cette valeur maximal et les coordonnées correspondantes.

Côté firmware, on modifie le code pour régulièrement lire ces 3 valeurs de registre, et les envoyer sur le canal UART.

Test : on reprend l'architecture complète, avec l'IP de binarisation modifiée. On teste cette architecture via Vitis, et on met en place un terminal pouvant recevoir la communication UART de la carte.

Le terminal reçoit de manière régulière des paramètres, mais ceux-ci sont très irréguliers et ne semble pas correspondre à des valeurs réelles.



```

PuTTY (inactive)
Binarisation : max value of 22 at x = 491 and y = 477
Binarisation : max value of 7 at x = 633 and y = 477
Binarisation : max value of 44 at x = 463 and y = 414
Binarisation : max value of 18 at x = 499 and y = 477
Binarisation : max value of 0 at x = 0 and y = 0
Binarisation : max value of 28 at x = 483 and y = 477
Binarisation : max value of 12 at x = 505 and y = 477
Binarisation : max value of 85 at x = 459 and y = 412
Binarisation : max value of 24 at x = 490 and y = 477
Binarisation : max value of 8 at x = 632 and y = 477
Binarisation : max value of 51 at x = 462 and y = 414
Binarisation : max value of 20 at x = 498 and y = 477
Binarisation : max value of 5 at x = 637 and y = 477
Binarisation : max value of 29 at x = 476 and y = 477
Binarisation : max value of 14 at x = 504 and y = 477
Binarisation : max value of 85 at x = 459 and y = 412
Binarisation : max value of 25 at x = 487 and y = 477
Binarisation : max value of 9 at x = 541 and y = 477
Binarisation : max value of 53 at x = 461 and y = 414
Binarisation : max value of 21 at x = 496 and y = 477
Binarisation : max value of 6 at x = 636 and y = 477
Binarisation : max value of 30 at x = 471 and y = 477
Binarisation : max value of 17 at x = 500 and y = 477

```

Ma solution pour lire les valeurs calculées par l'IP, ou bien mon calcul interne à l'IP ont l'air de rencontrer une erreur.

En l'état, **[PL-IP-010]** **semble non validée**, mais la solution côté PS **[PS-003]** **semble fonctionner correctement**.